ЗВІТ

до практичного заняття №8

Тема: Проєктування і опрацювання програми з обробленням виняткових ситуацій

Виконав: Вівчар Вадим Вікторович

Група: АЛК-43

Факультет: Інженерія програмного забезпечення

1. Мета роботи

Метою практичного заняття є засвоєння поняття виняткової ситуації та засобів її перехоплення, а також набуття практичних навичок програмної реалізації оброблення виняткових ситуацій у мові C++.

2. Теоретичні відомості

Виняткові ситуації в програмуванні виникають у разі появи непередбачених помилкових умов під час виконання програми. У мові C++ для роботи з такими ситуаціями використовується механізм оброблення винятків, який дає змогу не лише виявити помилку, а й коректно її обробити без аварійного завершення всієї програми. Основними елементами цього механізму є ключові слова try, catch і throw.

Блок try містить код, під час виконання якого може виникнути помилка. Якщо виняткова ситуація виникає, вона генерується оператором throw. Далі виконання передається до відповідного блока catch, який перехоплює виняток певного типу та виконує його оброблення. Із одним блоком try може бути пов’язано кілька блоків catch, а вибір конкретного обробника залежить від типу згенерованого винятку.

У C++ також можна використати форму catch(...), яка перехоплює будь-які виняткові ситуації незалежно від їх типу. Такий підхід є зручним для забезпечення загального захисту програми від аварійного завершення у випадках, коли тип винятку заздалегідь невідомий або коли потрібно гарантовано перехопити всі необроблені помилки.

У цьому практичному занятті розглядається клас Vect, призначений для роботи з одновимірними масивами типу int. У початковій реалізації для зберігання розміру масиву використовувалося поле типу char, що й стало джерелом помилки під час спроби створення об’єкта Vect a1(200). Помилку потрібно було виправити, не змінюючи тип поля char, а також реалізувати оброблення винятків у конструкторі, під час доступу до елементів масиву та в деструкторі класу.

Окремо важливо, що жоден виняток не повинен виходити за межі деструктора. Для виконання цієї вимоги рекомендується не генерувати винятки безпосередньо в тілі деструктора, а дії зі звільнення ресурсів винести в окремий метод, наприклад Destroy(), який викликається з деструктора всередині блока try/catch.

3. Хід роботи

Спочатку було проаналізовано початковий варіант програми, у якому оголошено клас Vect із полем char size, перевантаженим оператором доступу до елементів operator[] і методом Print(). У функції main() створювалися два об’єкти: Vect a(3) та Vect a1(200). Початковий код програми наведено на рисунку 1.

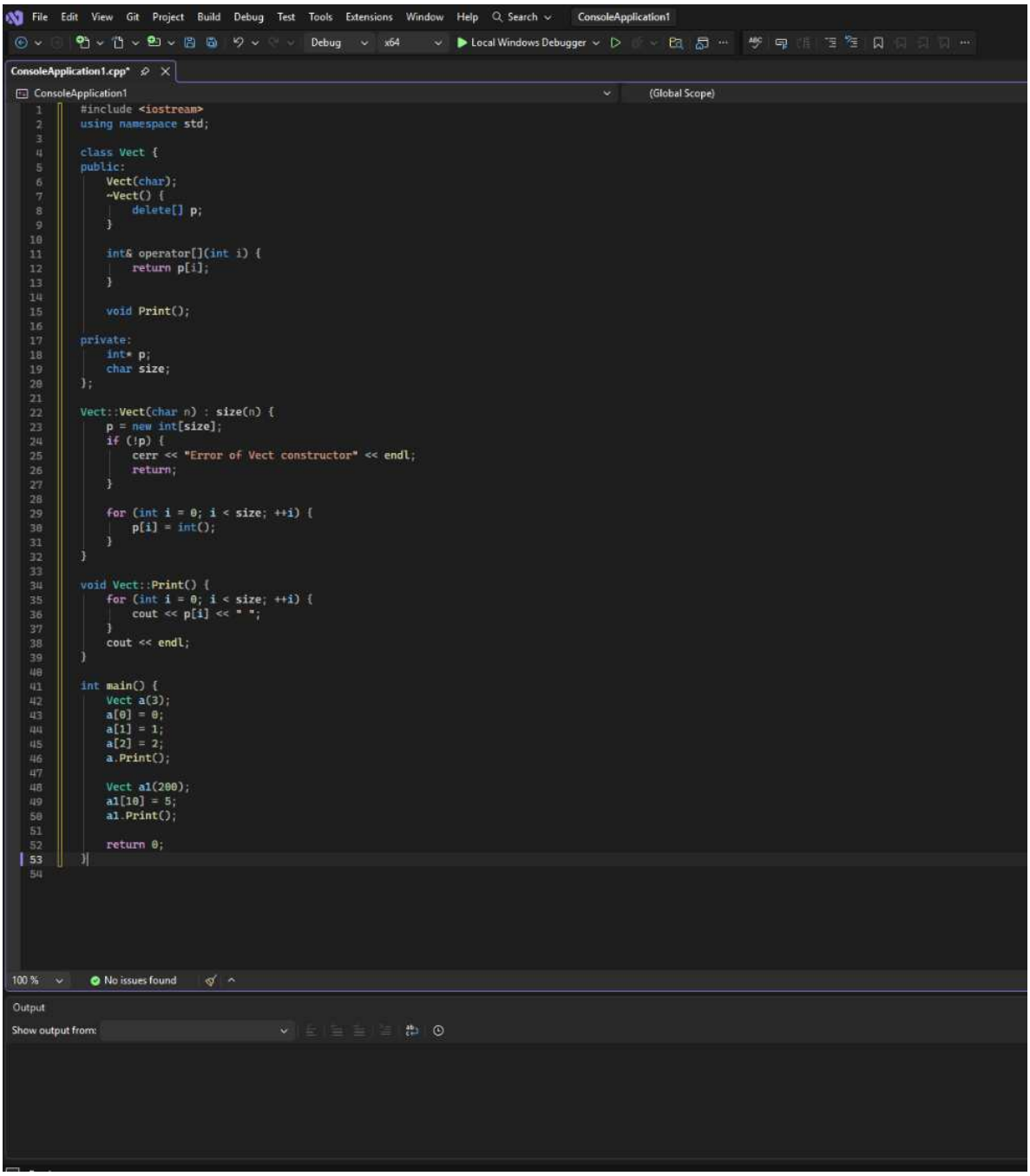


Рисунок 1 – Початковий код програми класу Vect

Після запуску початкового варіанта програми було встановлено, що створення першого об’єкта Vect a(3) виконується коректно, а в консоль виводяться значення 0 1 2. Проте під час роботи з другим об’єктом Vect a1(200) нормальне виконання програми порушується. Результат початкового запуску наведено на рисунку 2.

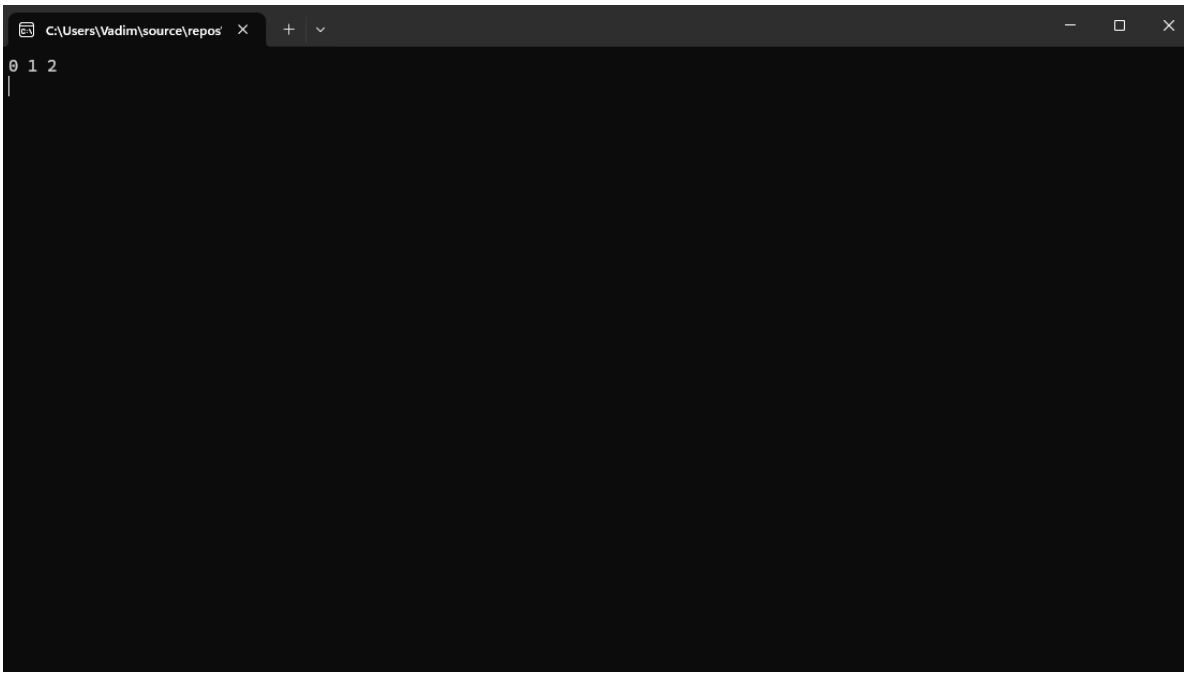


Рисунок 2 – Результат виконання початкової програми до моменту виникнення помилки

Для визначення причини несправності було виконано покрокове трасування програми у відладчику Visual Studio. Перед виконанням оператора Vect a1(200) було встановлено точку зупину, що дало змогу проаналізувати створення другого об’єкта поетапно. Цей момент наведено на рисунку 3.

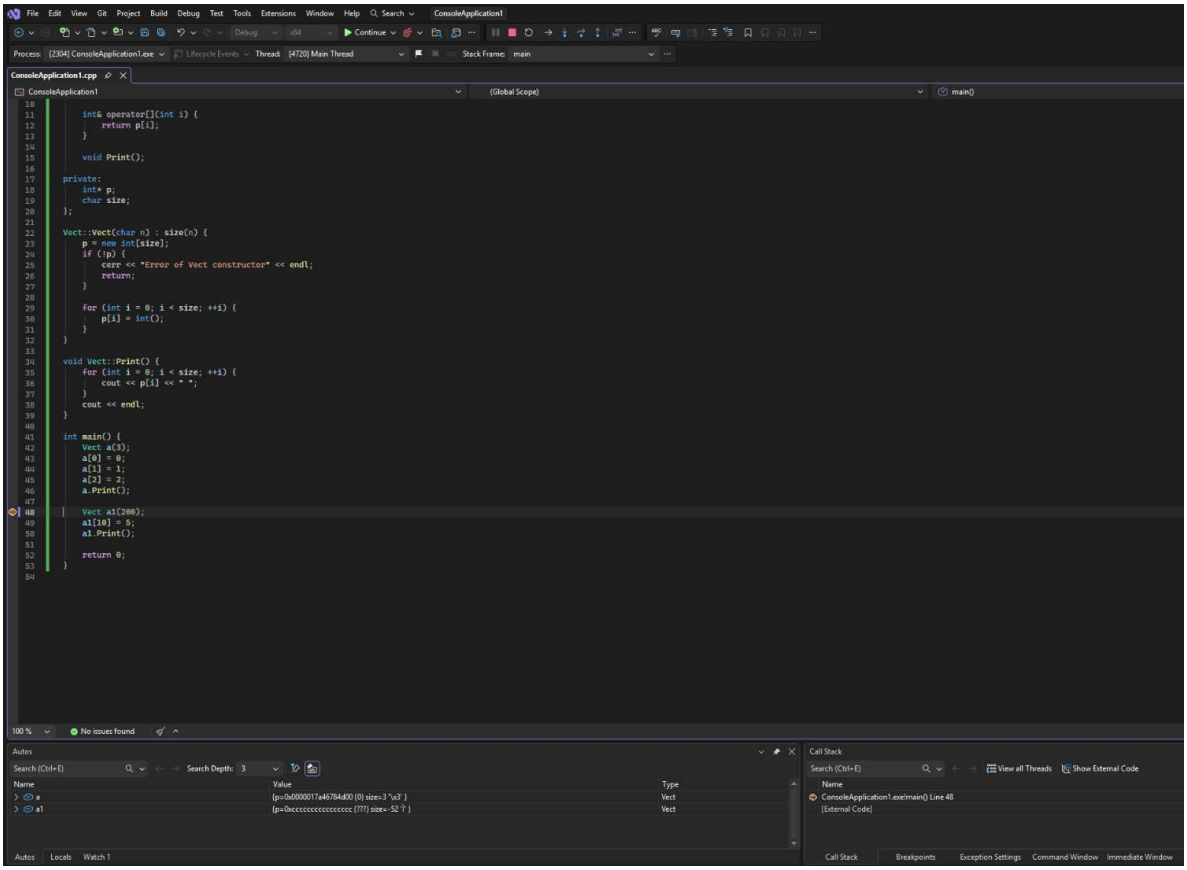


Рисунок 3 – Зупинка програми перед створенням об’єкта Vect a1(200)

Під час входу в конструктор було виявлено, що значення параметра n, яке передавалося як 200, у полі типу char інтерпретується як -56. Це пояснюється тим, що тип char фактично працює як signed char, тому число 200 виходить за межі діапазону -128 ... +127 і після перетворення сприймається як від’ємне. Відповідний етап відладки наведено на рисунку 4.

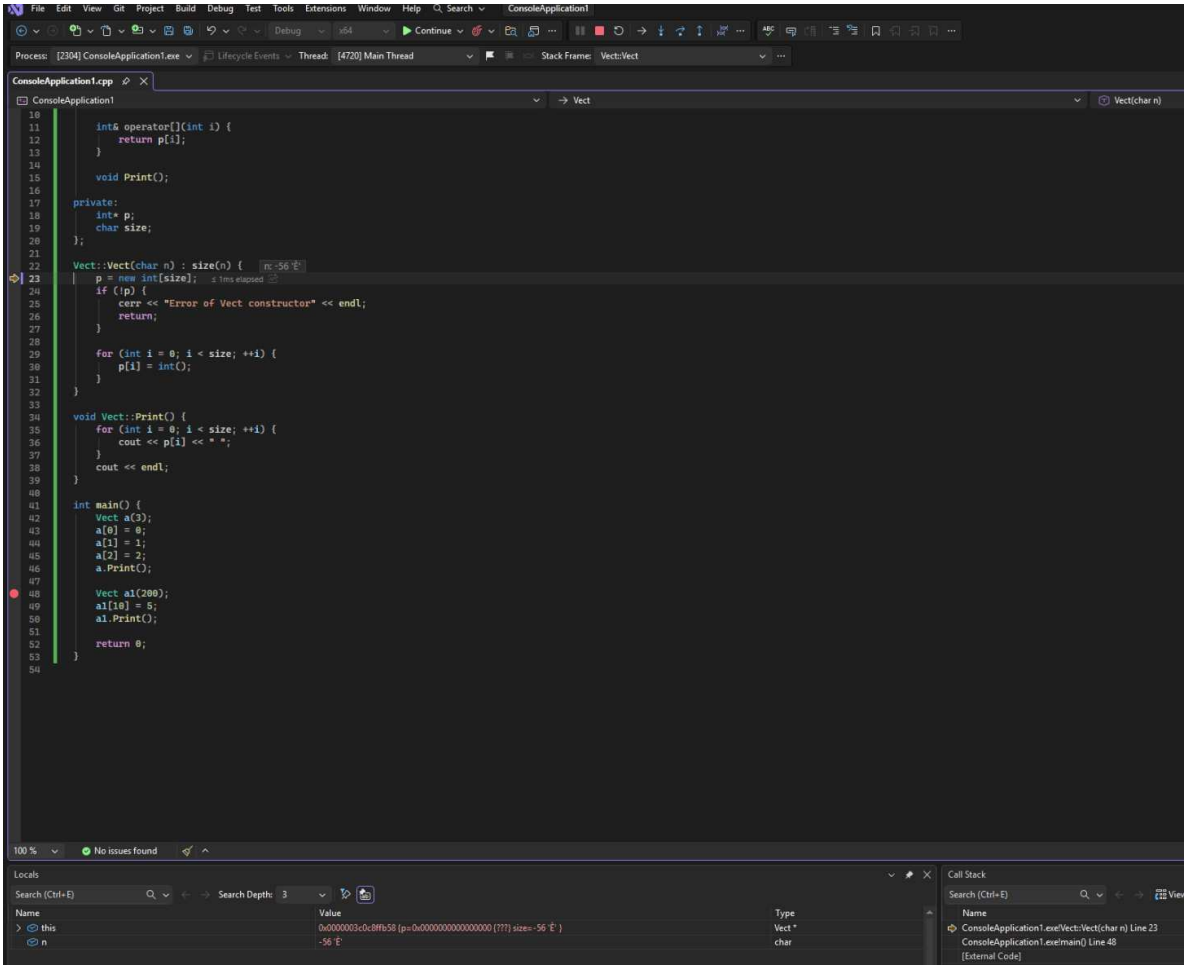


Рисунок 4 – Покрокове виконання програми в конструкторі класу Vect

Далі, під час виконання оператора виділення пам’яті new int[size], виникає виняткова ситуація std::bad_array_new_length, що підтверджує некоректність значення розміру масиву в початковій реалізації класу. Сам момент виникнення помилки наведено на рисунку 5.

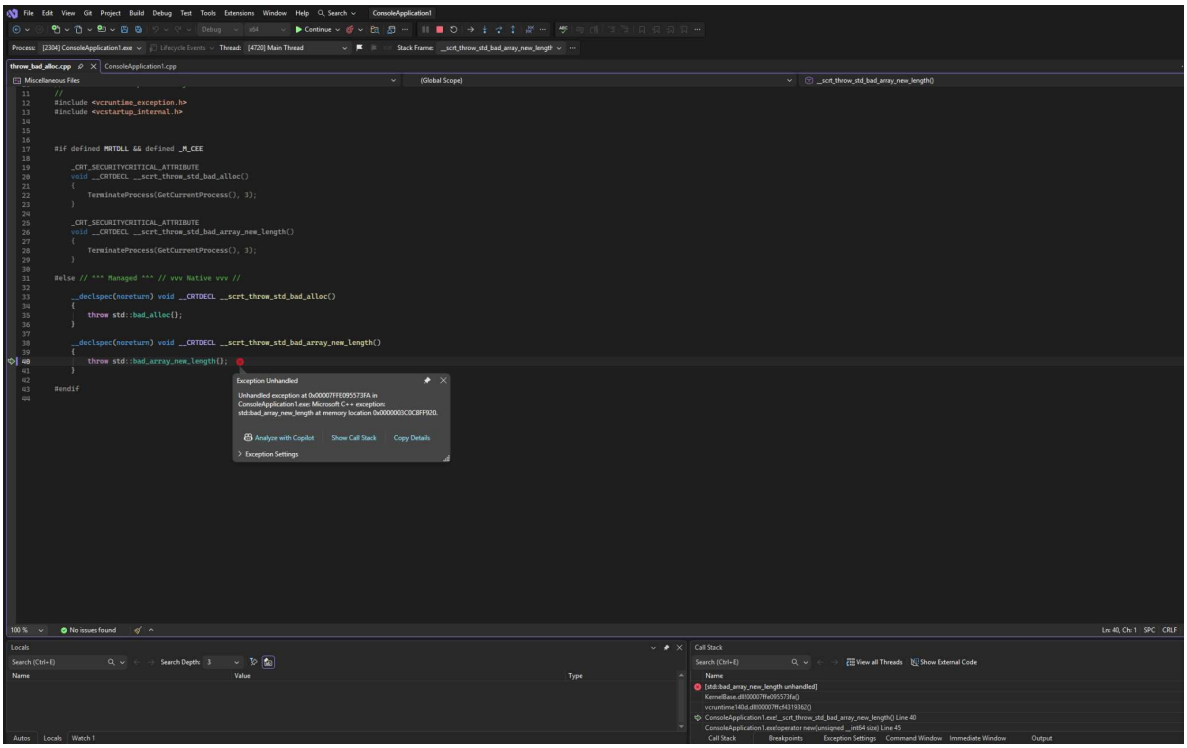


Рисунок 5 – Виникнення виняткової ситуації std::bad_array_new_length

Після аналізу було виконано виправлення програми без зміни типу поля char. Для цього конструктор було змінено так, щоб він приймав параметр типу int, перевіряв допустимість значення в межах від 0 до 255, а потім зберігав його у полі char size. Для коректного трактування реального розміру масиву було використано приведення до unsigned char. Виправлений конструктор наведено на рисунку 6.

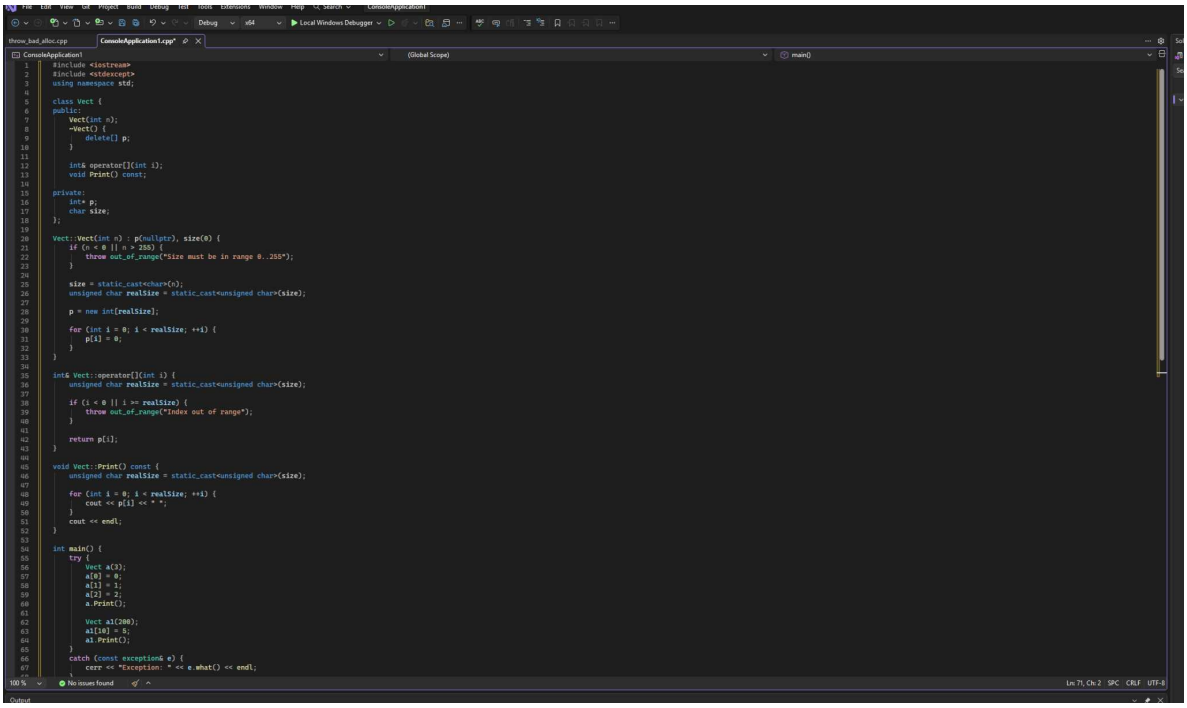


Рисунок 6 – Виправлений конструктор класу Vect

Далі було вдосконалено перевантажений оператор []. У ньому додано перевірку меж індексу, а у випадку виходу індексу за допустимий діапазон реалізовано генерацію винятку out_of_range. Це унеможливлює некоректний доступ до елементів масиву та дозволяє коректно обробляти помилки звернення до пам’яті. Відповідний фрагмент коду наведено на рисунку 7.

```
22            throw out_of_range("Size must be in range 0..255");
23        }
24
25        size = static_cast<char>(n);
26        unsigned char realSize = static_cast<unsigned char>(size);
27
28        p = new int[realSize];
29
30        for (int i = 0; i < realSize; ++i) {
31            p[i] = 0;
32        }
33    }
34
35    int& Vect::operator[](int i) {
36        unsigned char realSize = static_cast<unsigned char>(size);
37
38        if (i < 0 || i >= realSize) {
39            throw out_of_range("Index out of range");
40        }
41
```

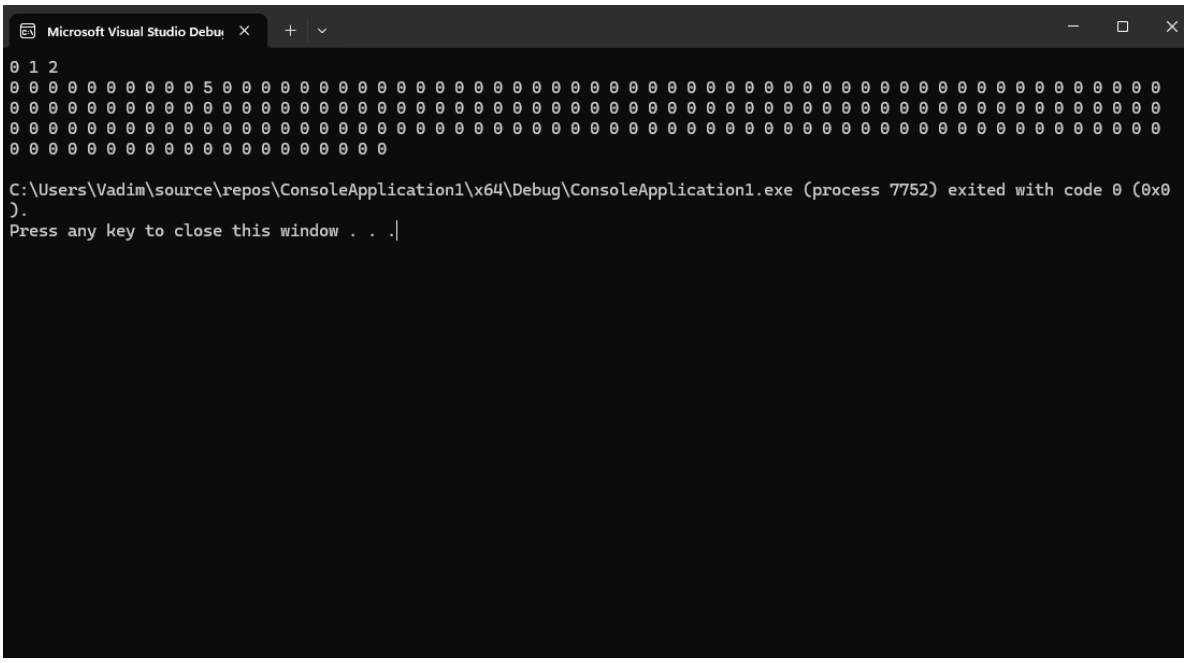
Рисунок 7 – Перевірка меж у перевантаженому операторі []

У функції main() було додано блок try/catch, який забезпечує централізоване перехоплення виняткових ситуацій, що можуть виникати під час створення об’єктів класу Vect та звернення до елементів масиву. Основну частину програми з реалізованим обробленням винятків наведено на рисунку 8.

```
39            throw out_of_range("Index out of range");
40        }
41
42        return p[i];
43    }
44
45    void Vect::Print() const {
46        unsigned char realSize = static_cast<unsigned char>(size);
47
48        for (int i = 0; i < realSize; ++i) {
49            cout << p[i] << " ";
50        }
51        cout << endl;
52    }
53
54    int main() {
55        try {
56            Vect a(3);
57            a[0] = 0;
58            a[1] = 1;
59            a[2] = 2;
60            a.Print();
61
62            Vect a1(200);
63            a1[10] = 5;
64            a1.Print();
65        }
66        catch (const exception& e) {
67            cerr << "Exception: " << e.what() << endl;
68        }
69
70        return 0;
00 %    No issues found
Output
```

Рисунок 8 – Основна частина програми з блоком try/catch

Після внесення зазначених змін програму було повторно запущено. Результати показали, що об’єкт Vect a1(200) тепер створюється коректно, запис у десятий елемент масиву виконується без помилок, а виведення елементів масиву відбувається штатно. Програма завершується без аварійного завершення. Результат успішного запуску наведено на рисунку 9.


Microsoft Visual Studio Debu
0 1 2
0 0 0 0 0 0 0 0 0 0 5 0 0 0 0 0 0 0 0 0

на рисунку 11.

```
38            p = nullptr;
39        }
40    }
41
42    Vect::~Vect() {
43        try {
44            Destroy();
45        }
46        catch (...) {
47            cerr << "Exception caught in destructor" << endl;
48        }
49    }
50
51    int& Vect::operator[](int i) {
52        unsigned char realSize = static_cast<unsigned char>(size);
53
54        if (i < 0 || i >= realSize) {
55            throw out_of_range("Index out of range");
56        }
57
58        return p[i];
59    }
60
61    void Vect::Print() const {
62        unsigned char realSize = static_cast<unsigned char>(size);
63
64        for (int i = 0; i < realSize; ++i) {
65            cout << p[i] << " ";
66        }
67        cout << endl;
68    }
```

Рисунок 11 – Безпечний деструктор з обробленням виняткових ситуацій

Фінальний запуск програми підтвердив, що після додавання безпечного деструктора об’єкти класу Vect коректно створюються та знищуються, виняткові ситуації обробляються належним чином, а програма завершується без аварійного завершення. Фінальний результат роботи програми наведено на рисунку 12.

```
Microsoft Visual Studio Debu
0 1 2
0 0 0 0 0 0 0 0 0 0 5 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

C:\Users\Vadim\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 2020) exited with code 0 (0x0
).
Press any key to close this window . . .
```

Рисунок 12 – Фінальний результат роботи програми з обробленням виняткових ситуацій

4. Аналіз отриманих результатів

У процесі виконання практичного заняття було встановлено, що головна причина помилки в початковій програмі пов’язана не з самим оператором new, а з некоректним трактуванням значення 200, записаного в поле типу

char. Через це розмір масиву ставав від’ємним, що призводило до виняткової ситуації вже на етапі створення об’єкта.

Практична реалізація оброблення винятків показала, що використання перевірок у конструкторі, перевантаженому операторі [], та головній функції програми дає змогу своєчасно виявляти помилки, формувати зрозумілі повідомлення й уникати аварійного завершення програми. Додавання безпечного деструктора забезпечило правильне звільнення пам’яті та відповідність вимогам роботи.

5. Контрольні запитання

1. Що таке виняткова ситуація в програмуванні?
Виняткова ситуація — це непередбачена помилкова умова, яка виникає під час виконання програми. У C++ для таких ситуацій передбачено спеціальний механізм оброблення винятків, який дозволяє перехоплювати помилки та коректно на них реагувати.

2. Для чого в C++ використовуються try, catch і throw?
Блок try містить код, у якому може виникнути помилка. Оператор throw використовується для генерації виняткової ситуації, а catch — для її перехоплення та оброблення. Якщо з блоком try пов’язано кілька блоків catch, то виконується той, тип якого відповідає типу винятку.

3. Чому в початковій програмі значення 200 перетворилося на -56?
Тому що поле size мало тип char, який інтерпретувався як signed char. Число 200 виходить за допустимий діапазон такого типу, тому після збереження воно сприймалося як від’ємне значення -56. Саме це й стало причиною помилки під час виділення пам’яті.

4. Чому не можна допускати вихід винятків за межі деструктора?
Тому що деструктор повинен завершувати роботу об’єкта безпечно. Якщо виняток вийде за його межі, це може призвести до некоректного завершення програми або до ускладнення процесу звільнення ресурсів.

5. Для чого використовується catch(...)?
Конструкція catch(...) використовується для перехоплення всіх виняткових ситуацій незалежно від їх типу. Її зручно застосовувати як останній обробник, щоб запобігти аварійному завершенню програми через необроблений виняток.

6. Яке виправлення було внесено в програму без зміни типу char?
Конструктор було змінено так, щоб він приймав розмір як значення типу int, перевіряв його допустимість у межах 0..255, а для фактичної роботи з розміром використовувалося приведення до unsigned char. Це дозволило залишити поле size типу char, але усунути помилку під час створення масиву.

6. Висновок

У ході виконання практичного заняття було досліджено причину виникнення помилки в програмі класу Vect та реалізовано її усунення без зміни типу поля char. Встановлено, що джерелом несправності було некоректне трактування значення 200, яке після збереження в char перетворювалося на -56, що призводило до помилки під час виділення пам’яті. Також було реалізовано перевірку меж індексу, додано блоки try/catch для перехоплення винятків та створено безпечний деструктор із методом Destroy() noexcept. Отже, мети практичного заняття досягнуто: опрацьовано механізм виявлення, перехоплення та оброблення виняткових ситуацій у програмах мовою C++.